

Computer Science & Programming

Week 04 — Recursion

Edoardo Terzi (ISIN) & Ricardo Omar Chavez-Garcia (IDSIA)

Spring Semester 2026

Bachelor in Data Science — SUPSI

Today

In this lecture we introduce one of the most fundamental ideas in computer science:

Recursion.

We will study:

- what recursion is and why it works,
- how recursive calls are executed in memory,
- how to analyze the running time of recursive algorithms,
- when recursion is an appropriate design choice.

Recursion will be essential for trees, divide-and-conquer, and many advanced algorithms.

Learning objectives

After this lecture, you should be able to:

- define recursion precisely,
- distinguish base case and recursive case,
- trace the execution of a recursive function,
- explain how the call stack evolves,
- estimate the running time of simple recursive algorithms,
- compare recursive and iterative solutions.

These tools will be used throughout the remainder of the course.

Reading guide

Reference textbook:

- **Data Structures and Algorithms in Python**

Relevant sections for Week 04:

- Chapter 4 — Recursion
- Sections on recursive algorithm analysis

Pay particular attention to:

- the structure of recursive definitions,
- the use of base cases,
- examples of linear and binary recursion.

Foundations of Recursion

What is recursion?

Definition

Recursion is a technique in which a function solves a problem by calling itself on a **smaller instance of the same problem**.

Instead of solving the problem directly, the function:

- reduces the problem to a simpler one
- calls itself to solve that simpler problem
- combines the result to obtain the final answer

Many problems in computer science have a naturally recursive structure.

Recursive problems in everyday reasoning

Many problems are naturally defined in terms of smaller versions of themselves.

Examples:

- Russian dolls inside other dolls
- folders inside folders in a filesystem
- mathematical definitions such as factorial

Conceptually:

problem $\rightarrow$ smaller problem $\rightarrow$ smaller problem $\rightarrow \dots \rightarrow$ simplest case

Recursion stops when the simplest case is reached.

Two essential components of recursion

Every recursive definition has two parts.

Base case

- the simplest instance of the problem
- solved directly
- stops the recursion

Recursive case

- reduces the problem to a smaller one
- calls the same function again

Without a base case, recursion never terminates.

Structure of a recursive function

A recursive function typically follows this pattern:

```
def f(problem):  
  
    if base_case:  
        return simple_solution  
  
    else:  
        return f(smaller_problem)
```

Key principle:

Each recursive call must move the problem closer to the base case.

Example: Factorial (mathematical definition)

The factorial function is naturally recursive.

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n - 1)! & \text{if } n > 0 \end{cases}$$

This definition contains:

- **Base case:** $0! = 1$
- **Recursive case:** $n! = n \cdot (n - 1)!$

Each step reduces the problem size by one.

Factorial in Python

The Python implementation directly follows the mathematical definition.

```
1 def factorial(n):  
2  
3     if n == 0:  
4         return 1  
5  
6     else:  
7         return n * factorial(n-1)
```

The function calls itself until the base case is reached.

Visualizing recursive calls

Consider the call:

factorial(4)

The recursive expansion is:

$$\begin{aligned} \text{factorial}(4) &= 4 \cdot \text{factorial}(3) \\ &= 4 \cdot (3 \cdot \text{factorial}(2)) \\ &= 4 \cdot (3 \cdot (2 \cdot \text{factorial}(1))) \\ &= 4 \cdot (3 \cdot (2 \cdot (1 \cdot \text{factorial}(0)))) \end{aligned}$$

Linear recursion

In the factorial example, each call produces exactly **one recursive call**.

Call chain:

$$\text{factorial}(4) \rightarrow \text{factorial}(3) \rightarrow \text{factorial}(2) \rightarrow \text{factorial}(1) \rightarrow \text{factorial}(0)$$

This type of recursion is called:

Linear recursion

The number of recursive calls grows proportionally to n .

Common beginner mistakes

Typical errors when writing recursive functions:

- forgetting the base case
- writing a base case that is never reached
- not reducing the problem size
- creating infinite recursion

If the recursive call does not move toward the base case, the program will never terminate.

Incorrect recursion example

```
1 def bad_factorial(n):  
2  
3     if n == 0:  
4         return 1  
5  
6     else:  
7         return n * bad_factorial(n)
```

Problem:

- the argument never decreases
- the base case is never reached

Result:

RecursionError: maximum recursion depth exceeded

Exercise — Understanding recursion

Consider the function:

```
def mystery(n):  
  
    if n == 1:  
        return 1  
    else:  
        return 1 + mystery(n-1)
```

Answer the following:

- What does this function compute?
- What is the base case?
- What is the recursive case?

Solution — Understanding recursion

What does it compute?

$$\text{mystery}(n) = n$$

Why?

$$1 + 1 + 1 + \cdots + 1$$

added exactly n times.

Base case

$$n = 1$$

Recursive case

$$\text{mystery}(n) = 1 + \text{mystery}(n - 1)$$

This is another example of **linear recursion**.

The Call Stack and Execution Model

What happens during a function call?

When a function is called:

- memory is allocated for its local variables,
- parameters are stored,
- execution jumps into the function body.

When the function returns:

- its local memory is released,
- execution continues where the function was called.

Python manages function calls using the **call stack**.

The call stack

The **call stack** is a stack data structure that stores all active function calls.

Each function call creates a **stack frame**.

A stack frame contains:

- the function parameters
- the local variables
- the return point in the program

The most recent function call is always placed at the top.

LIFO behavior

The call stack follows the principle:

Last In, First Out (LIFO)

If function A calls B , and B calls C :

$$A \rightarrow B \rightarrow C$$

Then the return order is:

$$C \rightarrow B \rightarrow A$$

The last function called is the first one that returns.

Tracing a recursive call

Consider again the recursive factorial:

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

We now trace the execution of:

factorial(3)

Call expansion for factorial(3)

The recursive calls expand as:

$$\text{factorial}(3) \rightarrow \text{factorial}(2) \rightarrow \text{factorial}(1) \rightarrow \text{factorial}(0)$$

Each new call must wait until the smaller call is completed.

This means that several calls are active at the same time.

Stack growth

As recursion proceeds, new stack frames are pushed.

```
+-----+
| factorial(0) |
+-----+
| factorial(1) |
+-----+
| factorial(2) |
+-----+
| factorial(3) |
+-----+
```

The frame on the top is the one currently being executed.

Why calls must wait

The call `factorial(3)` cannot finish immediately because it needs:

$$3 \cdot \textit{factorial}(2)$$

But `factorial(2)` also cannot finish immediately because it needs:

$$2 \cdot \textit{factorial}(1)$$

And so on, until the base case is reached.

Recursion therefore has two phases:

- **descent**: recursive calls go deeper
- **return**: results are combined while unwinding

Return phase

Recursive code

```
1 def factorial(n):  
2  
3     if n == 0:  
4         return 1  
5  
6     return n * factorial(n-1)
```

Return values

$$\text{factorial}(0) = 1$$

$$\text{factorial}(1) = 1 \cdot 1 = 1$$

$$\text{factorial}(2) = 2 \cdot 1 = 2$$

$$\text{factorial}(3) = 3 \cdot 2 = 6$$

Call:

$$\text{factorial}(3)$$

Blue = value of the current call parameter.

Red = value returned by the previous recursive call.

Recursion depth

The **recursion depth** is the maximum number of active recursive calls at the same time.

For `factorial(n)`:

- the deepest call is `factorial(0)`
- the number of active calls is $n + 1$

So the recursion depth grows linearly with n .

Why the base case is essential

Without a base case:

- recursive calls never stop,
- the stack keeps growing,
- memory is eventually exhausted.

In Python, this produces:

```
RecursionError: maximum recursion depth exceeded
```

The base case prevents infinite stack growth.

Recursion vs iteration

Aspect	Recursion	Iteration
Control mechanism	function calls	loop
Memory usage	many stack frames	one frame
Readability	often elegant	often explicit
Risk	stack overflow	indexing error

Recursion can be clearer, but iteration often uses less memory.

Exercise

Consider the function:

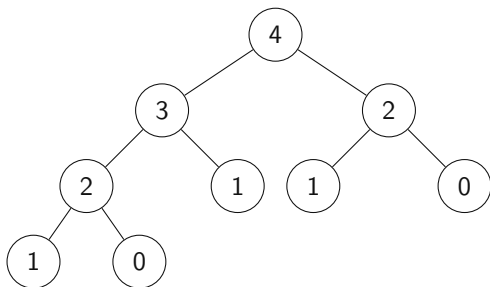
```
1 def mystery(n):  
2     if n <= 1:  
3         return n  
4     else:  
5         return mystery(n-1) + mystery(n-2)
```

Answer the following:

- What value is returned by `mystery(4)`?
- How many recursive calls are executed?
- Is this recursion **linear**?

Solution - Binary recursion

Recursion tree for mystery(4)



Number of calls per level

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$$

Each level roughly doubles the number of calls.

Recursive rule

$$m(n) = m(n-1) + m(n-2)$$

Each call generates **two new calls**.

If the recursion depth is about n :

$$\text{calls} \approx 2^n$$

Time complexity

$$T(n) = O(2^n)$$

Divide and Conquer Recursion

Divide and Conquer

Many recursive algorithms follow a common strategy:

Divide and Conquer

The idea is to solve a problem in three steps:

1. **Divide** the problem into smaller subproblems
2. **Solve** the subproblems recursively
3. **Combine** the results

This strategy appears in many algorithms:

- Merge Sort
- Quick Sort
- Binary Search

Divide and Conquer structure

Original problem

n

Divide

$n/2$ $n/2$

Solve recursively

$T(n/2)$ $T(n/2)$

Combine

Merge the results of the two subproblems.

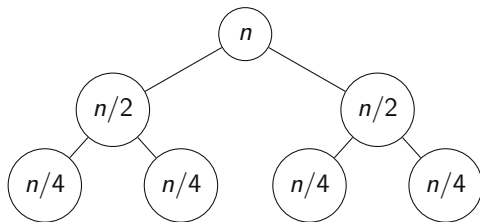
This leads to the recurrence:

$$T(n) = 2T(n/2) + f(n)$$

where $f(n)$ is the cost of the combine step.

Recursion tree in divide and conquer

Example: repeatedly splitting a problem in half.



Each level divides the problem into smaller pieces.

Example: Merge Sort

Merge Sort is a classic divide-and-conquer algorithm.

Goal:

- Sort a list of elements

Strategy:

1. Divide the list into two halves
2. Sort each half recursively
3. Merge the sorted halves

The algorithm repeatedly splits the input until small pieces remain.

Merge Sort — Python implementation

```
1 def merge_sort(arr):
2
3     if len(arr) <= 1:
4         return arr
5
6     mid = len(arr) // 2
7
8     left = merge_sort(arr[:mid])
9     right = merge_sort(arr[mid:])
10
11     return merge(left, right)
```

Each recursive call sorts a smaller part of the array.

The merge step

Goal

Combine two **sorted lists** into a single sorted list.

Example:

[1, 4, 7] [2, 3, 6]

Merge process:

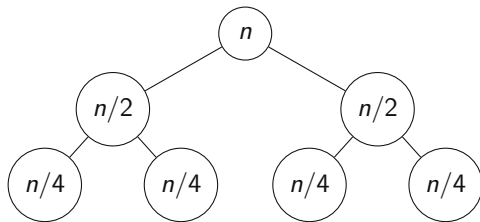
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7$

At each step we compare the **first elements** of the two lists and append the smallest one.

Python implementation

```
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(
5         right):
6         if left[i] < right[j]:
7             result.append(left[i])
8             i += 1
9         else:
10            result.append(right[j])
11            j += 1
12    result.extend(left[i:])
13    result.extend(right[j:])
14    return result
```

Recursion tree of Merge Sort



The depth of the recursion is about:

$$\log_2 n$$

Time complexity of Merge Sort

At each level of the recursion tree:

$$O(n)$$

work is required to merge the elements.

Number of levels:

$$\log n$$

Total complexity:

$$T(n) = O(n \log n)$$

This is significantly faster than quadratic sorting algorithms.

Exercise — Recursive sum

Given the list:

$$data = [1, [2, 3], [4, [5, 6]], 7]$$

Write a recursive function:

```
1 def deep_sum(data):
```

that returns the sum of all numbers in the list.

Example:

$$deep_sum(data) = 28$$

Solution — Recursive sum

```
1 def deep_sum(data):  
2     total = 0  
3     for x in data:  
4         if isinstance(x, list):  
5             total += deep_sum(x)  
6         else:  
7             total += x  
8     return total
```

Recursion vs Iteration

Recursion vs Iteration

Many computational problems can be solved in two ways:

Recursion

- a function calls itself
- the call stack stores intermediate states
- often closer to the mathematical definition

Iteration

- uses loops
- no additional stack frames
- often more memory efficient

Both approaches can solve the same problem.

Example problem: generating subsets

Example set:

[1, 2, 3]

Subsets:

[]

[1]

[2]

[3]

[1, 2]

[1, 3]

[2, 3]

[1, 2, 3]

For a set of size n :

Number of subsets:

$$2^n$$

Each element creates two choices:

- exclude the element
- include the element

We will solve this problem using:

- recursion
- iteration

Recursive solution

```
1 def generate_subsets(nums):
2     if not nums:
3         return [[]]
4     subsets = generate_subsets(nums[:-1]) # remove last element
5     new_subsets = []
6     for s in subsets:
7         new_subsets.append(s + [nums[-1]])
8     return subsets + new_subsets
```

Idea:

- generate subsets of the first $n - 1$ elements
- duplicate them
- add the last element to the duplicates

Recursive intuition

Each element creates two possibilities:

- exclude the element
- include the element

Example with three elements:

$$2 \times 2 \times 2 = 2^3$$

This forms a binary recursion tree.

Complexity of the recursive solution

Let n be the number of elements.

Number of subsets generated:

$$2^n$$

The algorithm must produce each subset.

Therefore:

$$T(n) = O(2^n)$$

This is an example of **exponential recursion**.

Iterative solution

```
1 def generate_subsets_iter(nums):
2
3     subsets = [[]]
4
5     for x in nums:
6         new_sets = []
7
8         for s in subsets: # Each iteration doubles the number of subsets
9             new_sets.append(s + [x])
10
11         subsets = subsets + new_sets
12
13     return subsets
```

Iterative intuition

Start with the empty subset:

$[\]$

Add element 1:

$[\], [1]$

Add element 2:

$[\], [1], [2], [1, 2]$

Each new element doubles the number of subsets.

Complexity of the iterative solution

At step k , the number of subsets is:

$$2^k$$

The algorithm processes all subsets.

Total running time:

$$T(n) = O(2^n)$$

Both recursive and iterative solutions have the same complexity.

Exercise

Consider the recursive function:

```
1 def f(n):  
2  
3     if n == 0:  
4         return 1  
5  
6     return f(n-1) + f(n-1)
```

Questions:

- What value does the function compute?
- What is the time complexity?

Solution

The recurrence is:

$$f(n) = 2 \cdot f(n - 1)$$

Therefore:

$$f(n) = 2^n$$

The recursion tree doubles at every level.

Time complexity:

$$T(n) = O(2^n)$$

Key Takeaways

- **Recursion** solves a problem by calling the same function on a smaller instance of the problem.
- Every recursive algorithm must contain:
 - a **base case**
 - a **recursive case**
- Recursive calls are executed using the **call stack**.
- Different recursion patterns produce different complexities:
 - linear recursion $\rightarrow O(n)$
 - divide & conquer $\rightarrow O(n \log n)$
 - exponential recursion $\rightarrow O(2^n)$